CS6910 : Fundamentals of Deep Learning

Assignment 1 Report

Team 6

Akash Kumar Paul(CS25S016)
Shrishti Bekal Mudiakal(CS25S009)

Professor: Dr. C. Chandra Sekhar

Department of Computer Science & Engineering
Indian Institute of Technology Madras

Date: 03-03-2026

Contents

1 Task 1: Comparison of Optimization Methods

1.1 Model Configuration

- Architecture: MLFFNN (20–10 hidden nodes)
- Activation: Tanh
- Loss: Cross-Entropy
- Learning Mode: Pattern Mode
- Stopping Criterion: Change in average error below threshold
- Same learning rate and same weight initialization

1.2 Optimization Methods

- Delta Rule
- Generalized Delta Rule
- AdaGrad
- RMSProp
- Adam

1.2.1 Delta Rule

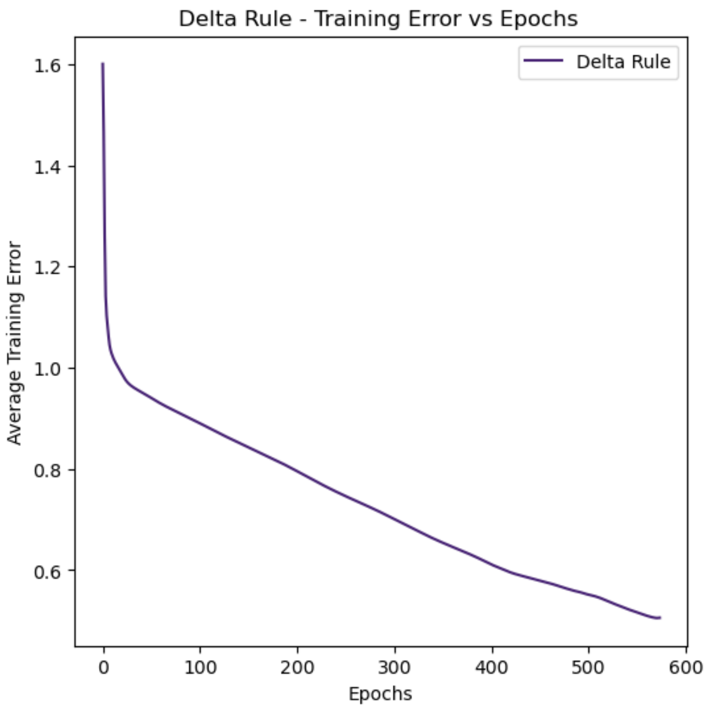
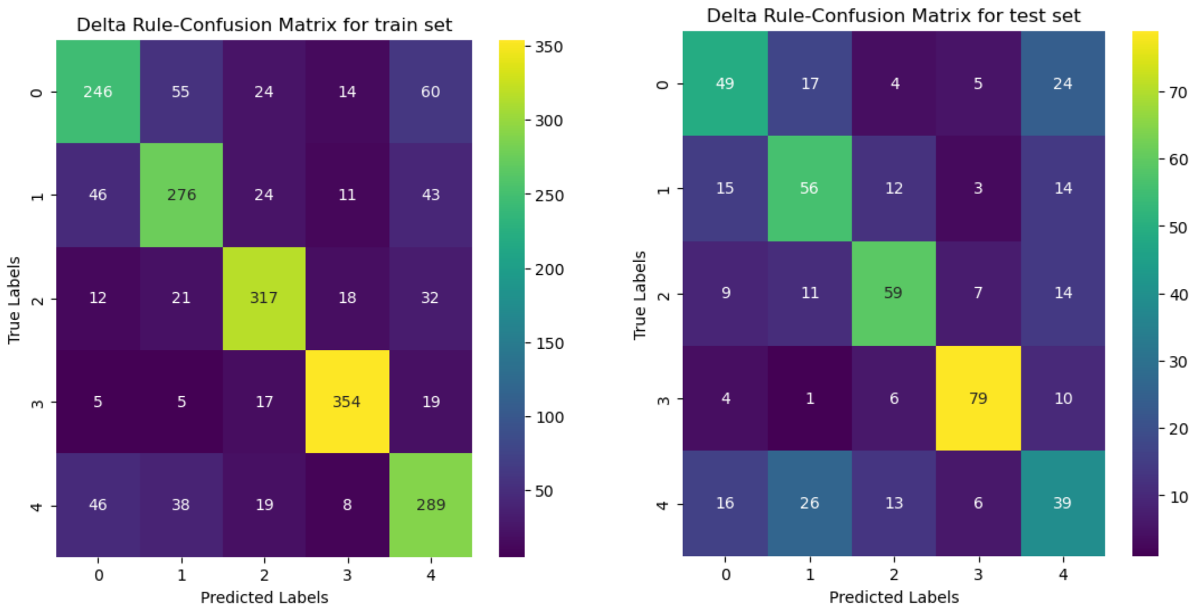


Figure 1: Delta Rule – Training Error vs Epoch



(a) Training Confusion Matrix (b) Test Confusion Matrix

Figure 2: Delta Rule Confusion Matrices

1.2.2 Generalized Delta Rule

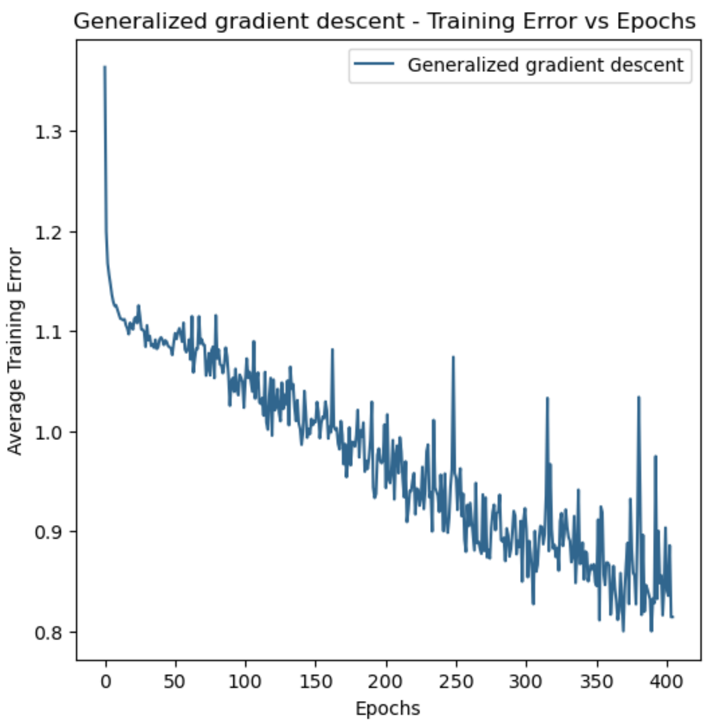
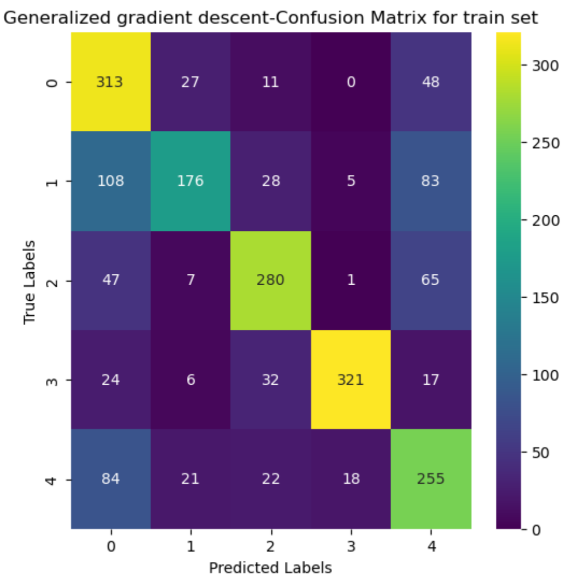
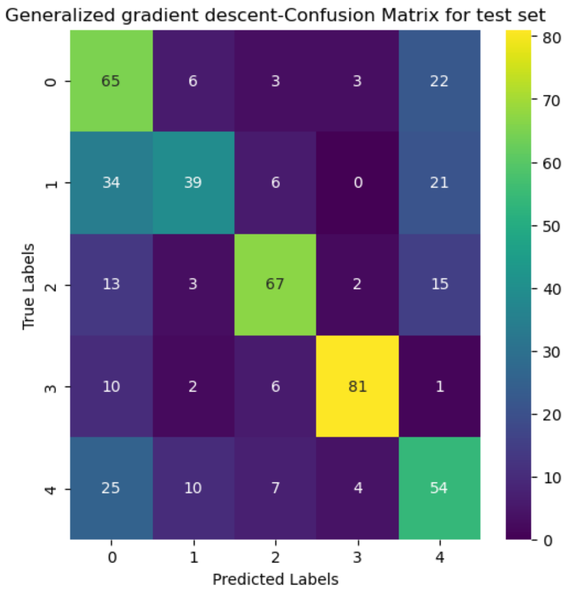


Figure 3: Generalized Delta Rule – Training Error vs Epoch



(a) Training Confusion Matrix



(b) Test Confusion Matrix

Figure 4: Generalized Delta Rule Confusion Matrices

1.2.3 AdaGrad

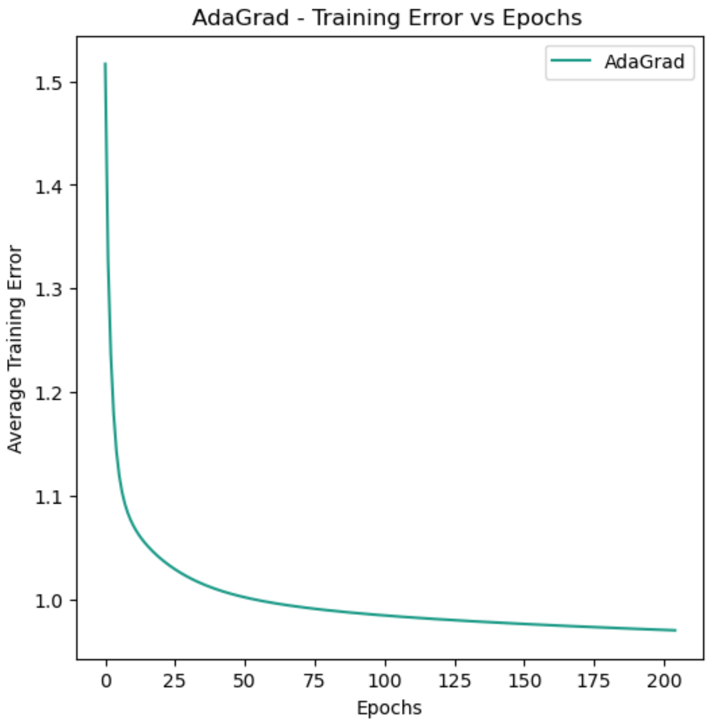
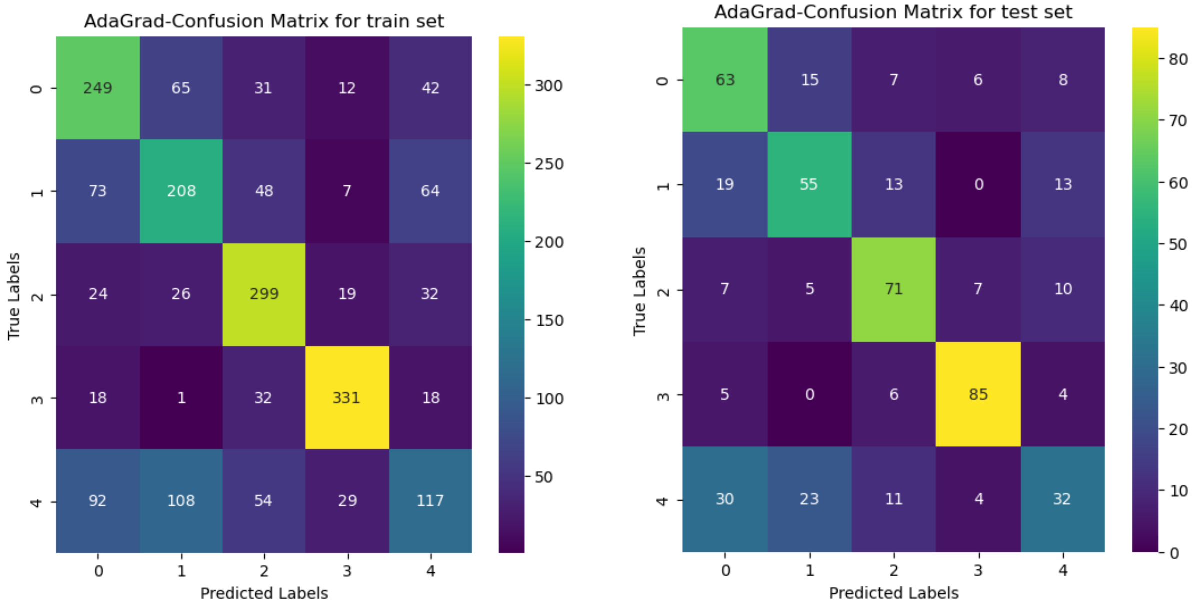


Figure 5: AdaGrad – Training Error vs Epoch



(a) Training Confusion Matrix (b) Test Confusion Matrix

Figure 6: AdaGrad Confusion Matrices

1.2.4 RMSProp

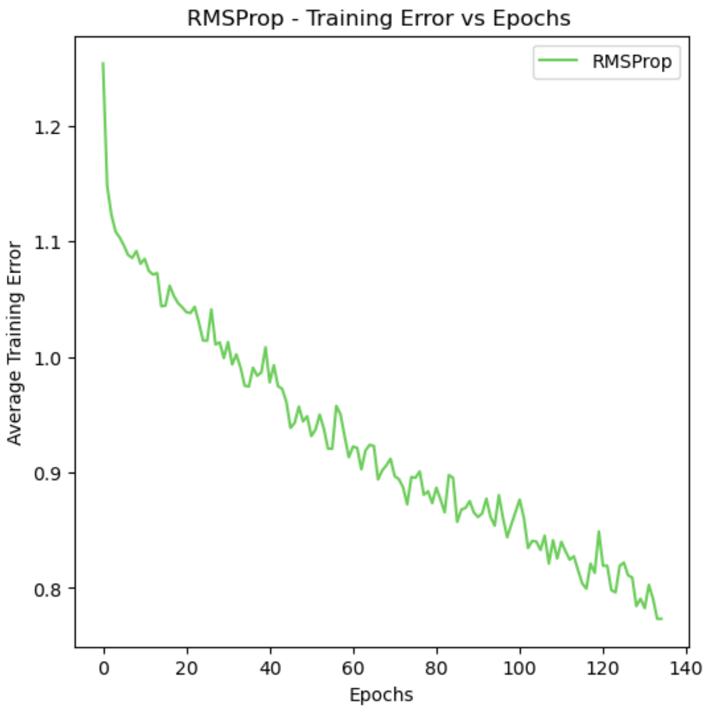
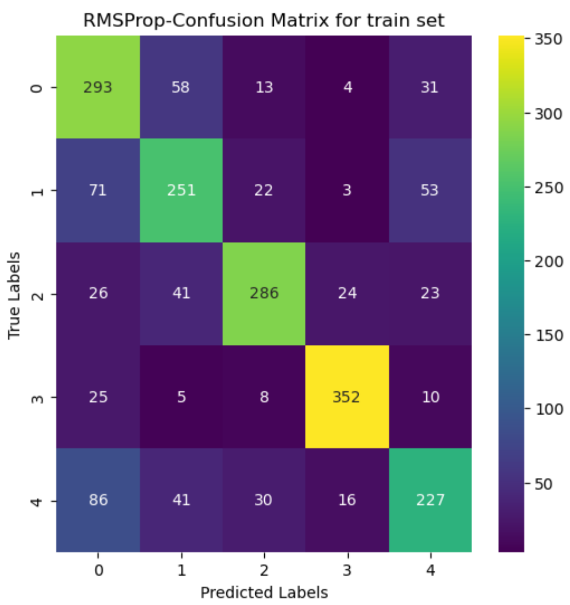
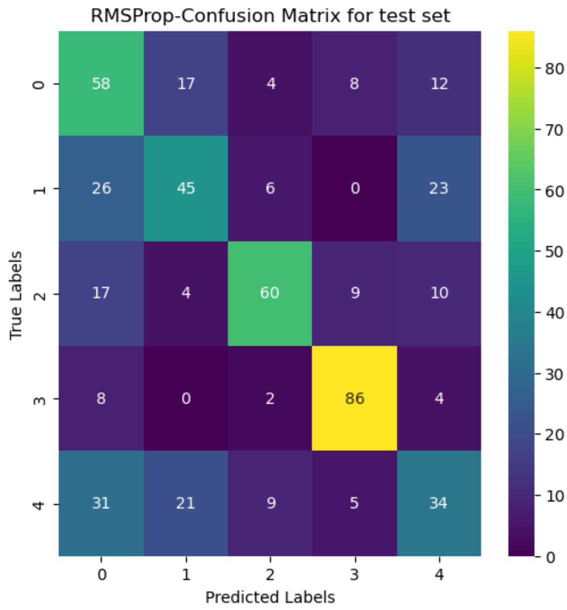


Figure 7: RMSProp – Training Error vs Epoch



(a) Training Confusion Matrix



(b) Test Confusion Matrix

Figure 8: RMSProp Confusion Matrices

1.2.5 Adam

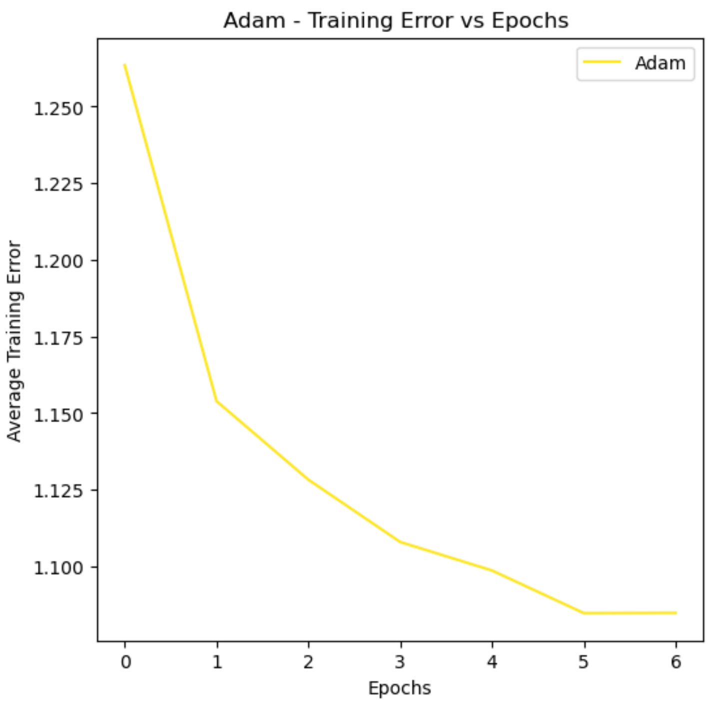
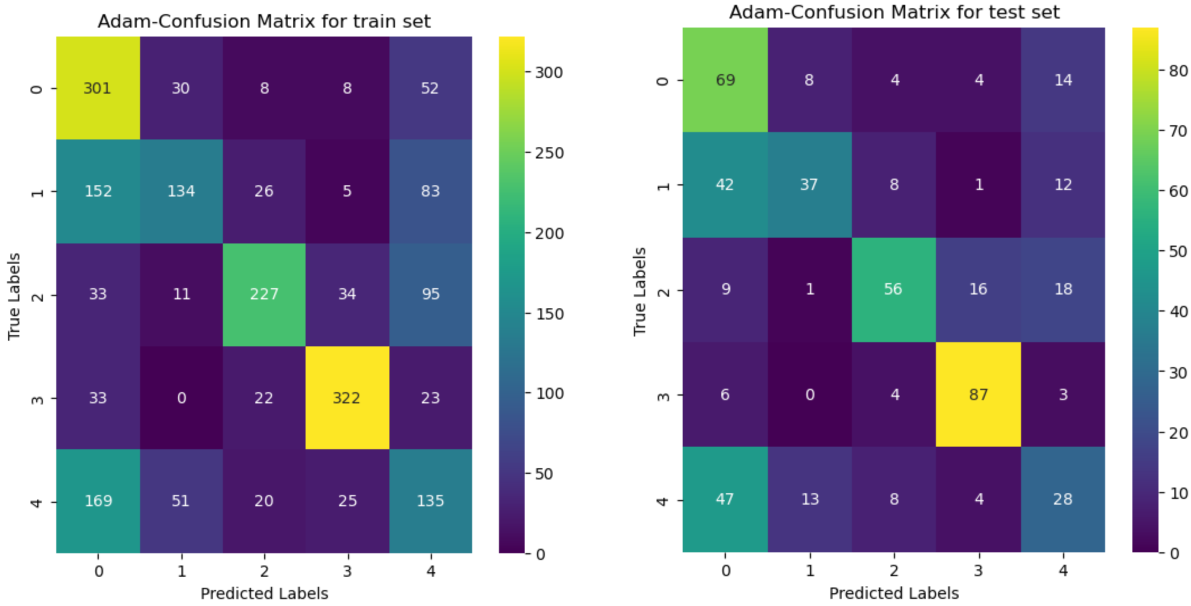


Figure 9: Adam – Training Error vs Epoch



(a) Training Confusion Matrix (b) Test Confusion Matrix

Figure 10: Adam Confusion Matrices

1.3 Comparison of Convergence

Optimizer	Epochs to Converge
Delta Rule	573
Generalized Delta	404
AdaGrad	204
RMSProp	134
Adam	6

Table 1: Comparison of Optimizers

1.4 Observation

Adam converges the fastest among all the optimizers, followed by RMSProp and AdaGrad. The Delta Rule and Generalized Delta Rule take more epochs to converge. Overall, adaptive optimizers perform better and reach convergence much quicker than the traditional gradient-based methods.

2 Task 2: Comparison of Normalization Methods

2.1 Model Configuration

- Architecture: MLFFNN (20–10 hidden nodes)
- Activation: Tanh
- Loss: Cross-Entropy
- Learning Mode: Mini-batch
- Weight Update: Adam

2.1.1 No Normalization

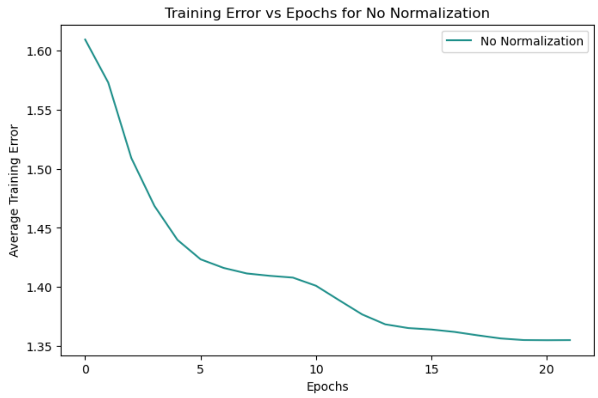
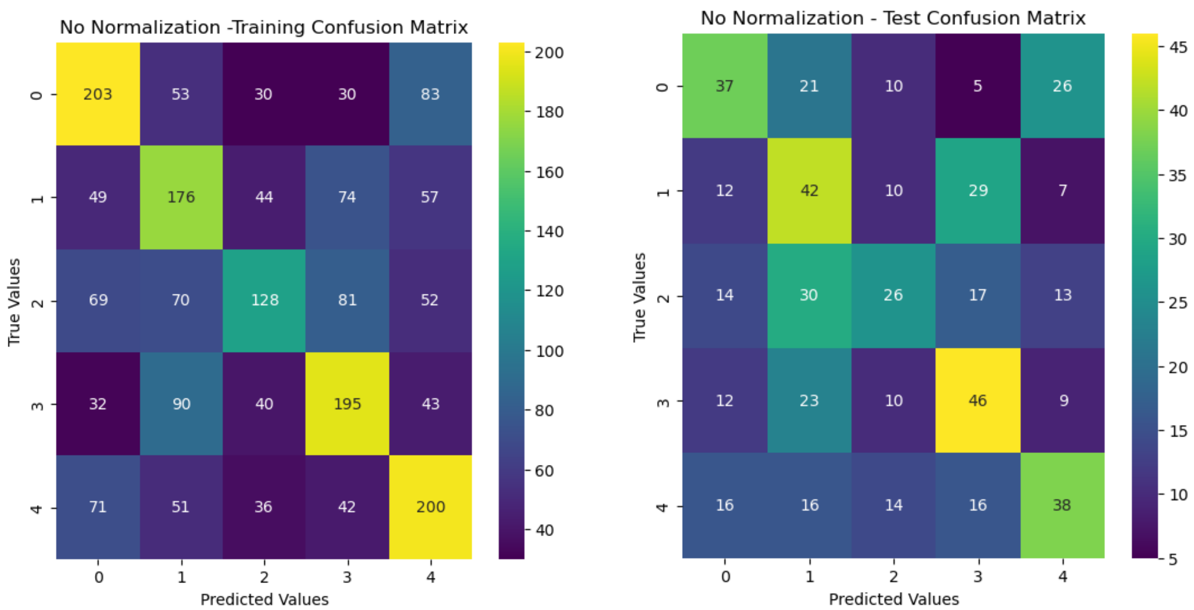


Figure 11: No Normalization – Training Error vs Epoch



(a) Training Confusion Matrix

(b) Test Confusion Matrix

Figure 12: No Normalization Confusion Matrices

2.1.2 Layer Normalization

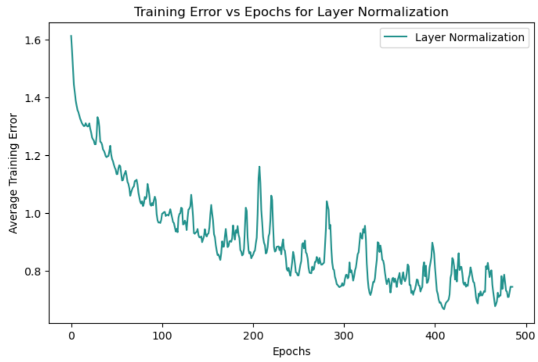
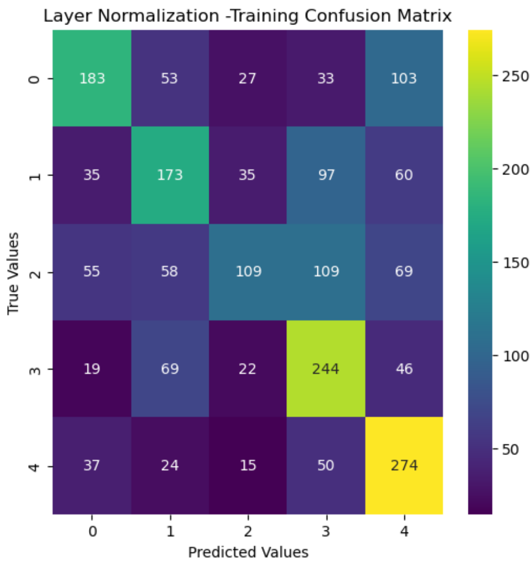
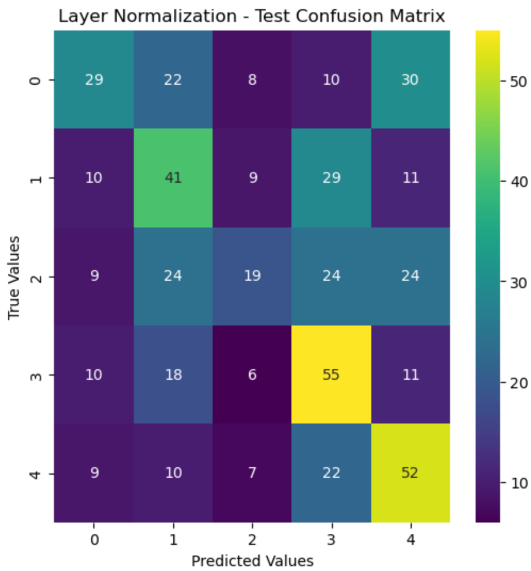


Figure 13: Layer Normalization – Training Error vs Epoch



(a) Training Confusion Matrix



(b) Test Confusion Matrix

Figure 14: Layer Normalization Confusion Matrices

2.1.3 Batch Normalization

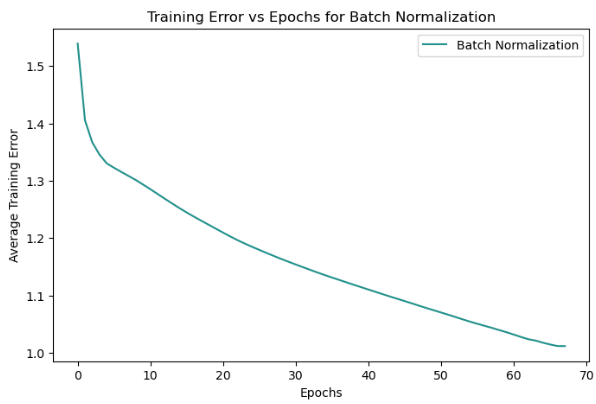
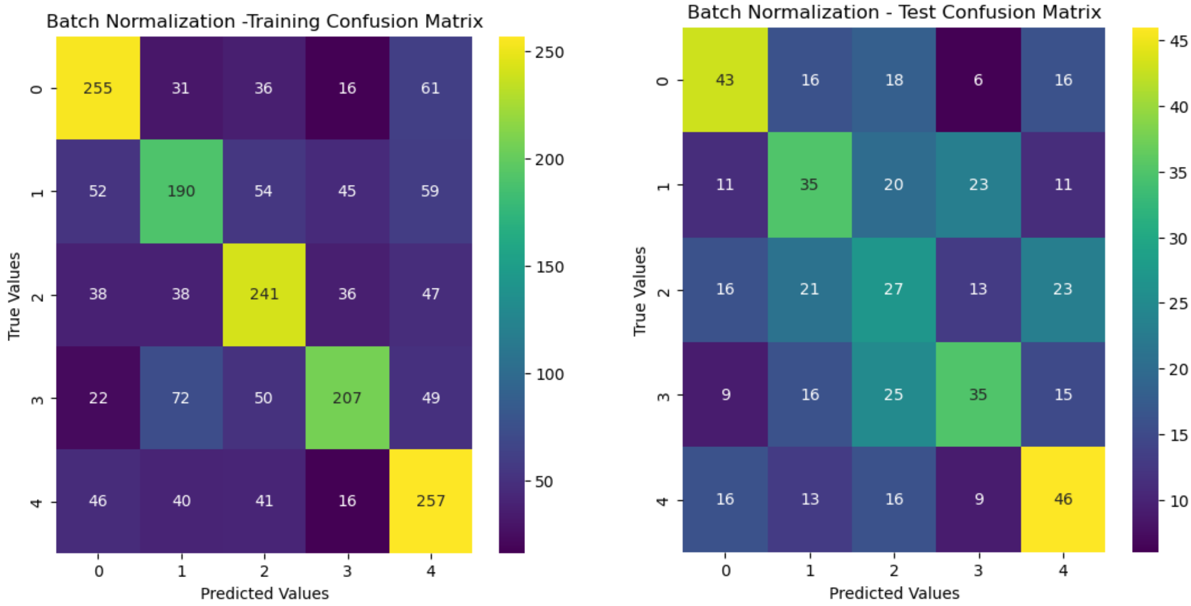


Figure 15: Batch Normalization – Training Error vs Epoch



(a) Training Confusion Matrix

(b) Test Confusion Matrix

Figure 16: Batch Normalization Confusion Matrices

2.2 Comparison of Convergence

Normalization	Epochs to Converge
No Norm	331
Layer Norm	90
Batch Norm	202

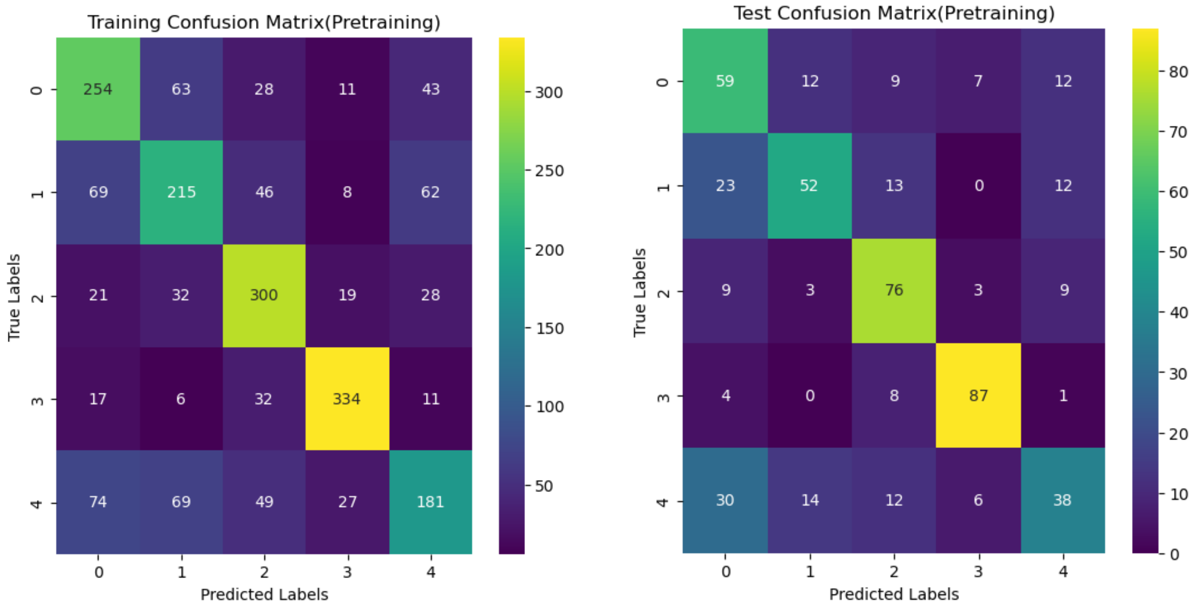
Table 2: Comparison of Normalization Methods

2.3 Observation

Layer Normalization converges the fastest among all the normalization methods, followed by Batch Normalization. No Normalization takes the highest number of epochs to converge. Overall, applying normalization improves convergence speed compared to training the neural network without normalization.

3 Task 3: Stacked Autoencoder

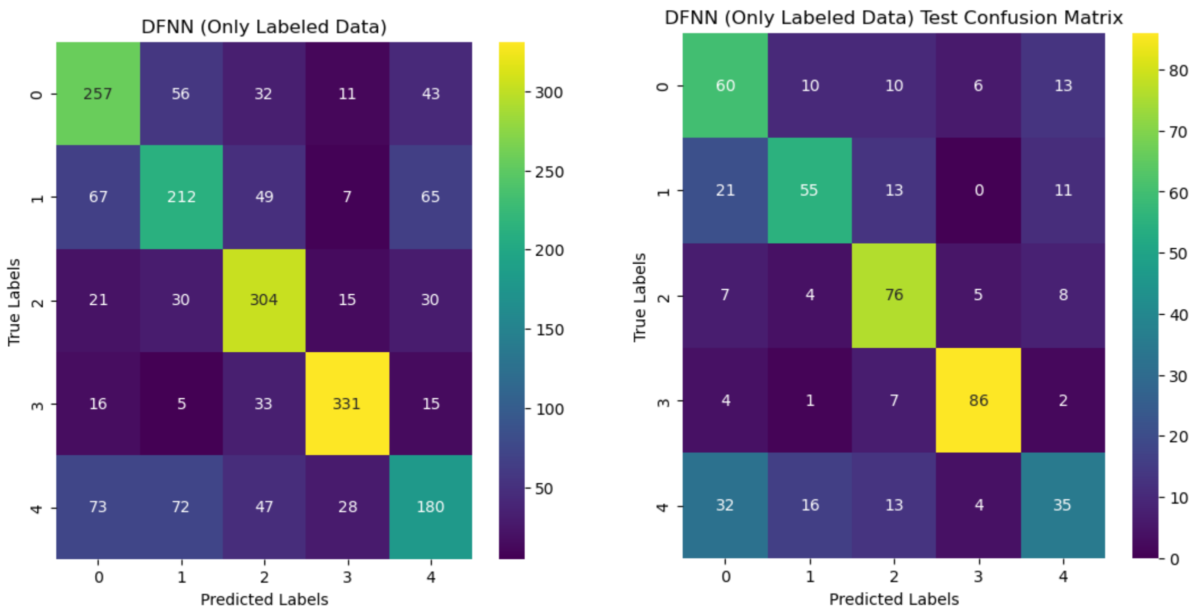
3.1 DFNN (Trained with Labeled Data)



(a) Training Confusion Matrix (b) Test Confusion Matrix

Figure 17: DFNN Trained Only with Labeled Data

3.2 DFNN (Stacked Autoencoder Pretrained using Unlabeled + Finetuned using Labeled Data)



(a) Training Confusion Matrix (b) Test Confusion Matrix

Figure 18: DFNN with Stacked Autoencoder Pretraining

3.3 Observation

The DFNN trained only with labeled data achieves higher training accuracy but tends to overfit. The stacked autoencoder pretrained DFNN shows slightly lower training accuracy with more balanced class predictions. This indicates that unsupervised pretraining helps reduce overfitting and improves generalization performance on unseen data.

*****